MASTERARBEIT | MASTER'S THESIS

Titel | Title

Termination results for hybrid approach of Joint Spectral Radius computation

verfasst von | submitted by

Aaron Pumm, B.Sc.

angestrebter akademischer Grad | in partial fulfilment of the requirements for the degree of

Master Zusatz M.Sc.

Wien, | Vienna, 2026

Studienkennzahl lt. Studienblatt | degree programme code as it appears on the student record sheet:	UA 066 821
Studienrichtung lt. Studienblatt | degree programme as it appears on the student record sheet:	Masterstudium Mathematik
Betreut von | Supervisor:	Assoz. Prof. Dr. Martin Ehler
Ko-betreut von | Co-Supervisor:	Dr. Thomas Mejstrik MMag.

Contents

1. Introduction and Background **6**
1.1. Notation and Preliminaries . . . 7
1.2. Motivation and Applications of the Joint Spectral Radius . . . 7
1.3. Fundamental Theoretical Results . . . 9
1.4. Preprocessing and Normalization . . . 15

2. Invariant-Polytope Algorithm **17**
2.1. Overview . . . 17
2.2. Algorithmic Formulation . . . 18
2.3. Termination Theory . . . 19
2.4. Implementation Details . . . 22
2.5. Remarks . . . 23

3. Finite-Tree Algorithm **24**
3.1. Overview . . . 24
3.2. Algorithmic Formulation . . . 26
3.3. Termination Theory . . . 27
3.4. Implementation Details . . . 30
3.5. Remarks . . . 30

4. Invariant-Tree Algorithm **31**
4.1. Overview . . . 31
4.2. Algorithmic Formulation . . . 32
4.3. Termination Theory . . . 33
4.4. Implementation Considerations . . . 37
4.5. Remarks . . . 38

5. Numerical Testing **40**
5.1. Experimental Setup . . . 40
5.2. Generated Matrix Families . . . 40
5.3. Instances with a Unique s.m.p. . . . 41
5.4. Instances with Multiple s.m.p.s . . . 42

6. Conclusions and Outlook **44**
6.1. Summary of Invariant-Tree Numerics . . . 44
6.2. Resolved Problem Classes . . . 44

6.3. Remaining Open Problem Classes 45
6.4. Directions for Future Research 45

A. Notation Glossary **47**
A.1. Constants and Globally Defined Variables 47
A.2. Products and Multi-Indices 47
A.3. Norms and JSR 48
A.4. Algorithmic Notation 48
A.5. Tree Terminology 49

Zusammenfassung

Diese Arbeit behandelt die Berechnung des gemeinsamen Spektralradius (JSR) einer endlichen Menge von Matrizen. Die Kapitel zum Invariant-Polytope-Algorithmus und zum Finite-Tree-Algorithmus fassen den bestehenden Forschungsstand zusammen und dienen als Grundlage für den hybriden Ansatz. Der originäre Beitrag dieser Arbeit liegt in der detaillierten Ausarbeitung des Invariant-Tree-Algorithmus (Algorithmus 3) sowie seiner Terminierungstheorie auf Basis von (Mejstrik and Reif, 2025). Implementierungsdetails zur endlichen 1-Beschränkung erzeugter Produktmengen werden aus der Literatur übernommen und nicht als neuer Beitrag beansprucht. Zusätzlich werden numerische JSR-Rechnungen für Matrizenmengen gezeigt, bei denen bestehende Methoden scheitern oder ineffizient sind.

Abstract

This thesis studies the computation of the joint spectral radius (JSR) for finite matrix families. The chapters on the invariant-polytope algorithm and the finite-tree algorithm are literature-based summaries that provide the theoretical background for the hybrid method. The original contribution of the thesis is the detailed formulation of the invariant-tree algorithm (Algorithm 3) and its termination theory, based on (Mejstrik and Reif, 2025). Implementation details for finite 1-bounding of generated product sets are adopted from the cited literature and are not claimed as original here. The thesis also includes numerical experiments on matrix families for which standalone methods fail or become computationally expensive.

Keywords: Joint spectral radius, invariant-tree algorithm, invariant-polytope algorithm, finite-tree algorithm, matrix norms.

1. Introduction and Background

The *joint spectral radius* (JSR) was introduced by G.-C. Rota and G. Strang in 1960 (Rota and Strang, 1960). It describes the maximal exponential growth rate of long matrix products drawn from a finite family. Since then, the JSR has become a central quantity in the study of discrete linear dynamics.

The JSR appears in several areas of mathematics and engineering. Important examples include wavelet theory and subdivision, where it enters the analysis of refinement schemes (Daubechies and Lagarias, 1992), and control theory, where it characterizes the stability of switched linear systems (Jungers, 2009). The computational difficulty of determining the JSR has also motivated the development of specialized algorithms, including the invariant-polytope algorithm (Guglielmi and Protasov, 2013) and the finite-tree algorithm (Möller and Reif, 2014).

Despite substantial progress, exact JSR computation remains difficult. The space of admissible products grows exponentially with the product length, and several hardness results show that no general efficient procedure should be expected.

This thesis has two roles. Chapters 2 and 3 summarize established algorithms and their termination mechanisms from the literature. The original contribution of the thesis is Chapter 4, which develops a detailed formulation of Algorithm 3 and its termination theory on the basis of (Mejstrik and Reif, 2025). Implementation details for finite 1-bounding of generated product sets are adopted from the cited work and are not claimed as original here.

Thesis Outline

Chapter 1 introduces the JSR, its basic properties, and the common preprocessing step used later. Chapter 2 presents the invariant-polytope algorithm as the simplest certificate mechanism in the thesis. Chapter 3 then develops the finite-tree approach as the first tree-based generalization of that idea. Chapter 4 contains the thesis-specific contribution: the invariant-tree algorithm and its termination theory. In related literature, the same construction is also called the *tree-flavoured-invariant-polytope-algorithm* (Mejstrik and Reif, 2025). Chapter 5 discusses numerical experiments, Chapter 6 closes with conclusions and future directions, and Appendix A collects the notation used throughout the thesis.

1.1. Notation and Preliminaries

Unless stated otherwise, all matrices are real $d \times d$ matrices and all matrix norms are submultiplicative. We use the following notation conventions consistently across all chapters:

- Matrix families and dimensions:

$$\mathcal{A} = \{A_1, \ldots, A_n\}, \qquad \tilde{\mathcal{A}} = \{\tilde{A}_1, \ldots, \tilde{A}_n\}, \qquad d = \text{matrix dimension}, \quad n = |\mathcal{A}|.$$

- Generator notation:

$$\mathcal{G} = \{G_1, \ldots, G_m\}, \qquad m = |\mathcal{G}|.$$

- Positive indices and products:

$$\mathcal{I} := \{1, \ldots, n\}, \quad I = [i_1, \ldots, i_k] \in \mathcal{I}^k, \quad A_I := A_{i_k} \cdots A_{i_1}, \quad \mathcal{A}^k := \{A_I : I \in \mathcal{I}^k\}.$$

- Scalars and spectra: α denotes a scalar, x a vector, λ an eigenvalue, $\rho(\cdot)$ the spectral radius, and $\sigma(\cdot)$ a singular value.
- Polytope norms:

$$\|\cdot\|_{\mathrm{co_s}(V)} = \|\cdot\|_V.$$

Appendix A contains the full notation glossary used throughout the thesis.

1.2. Motivation and Applications of the Joint Spectral Radius

Let $\mathcal{A} = \{A_1, \cdots, A_n\}$ be a finite set of $d \times d$ matrices. Consider a discrete-time linear system generated by a matrix sequence from $\mathcal{A}$:

$$x_{t+1} = B_t x_t, \quad x_0 \in \mathbb{R}^d, \tag{1.1}$$

where $B_t \in \mathcal{A}$ for all $t \in \mathbb{N}$.

A key question is whether the system is *uniformly asymptotically stable*, i.e., whether every trajectory (x_t) converges to zero for all initial states and all matrix sequences $(B_t)_{t\geq 0}$ with $B_t \in \mathcal{A}$:

$$\lim_{t\to\infty} x_t = 0.$$

This stability question is answered by a generalization of the spectral radius for a single matrix, the so-called *joint spectral radius (JSR)* of a matrix set $\mathcal{A}$ (Rota and Strang, 1960).

Definition 1.2.1. (Rota and Strang, 1960) For a finite set of matrices $\mathcal{A} = \{A_1, \ldots, A_n\}$, the JSR is defined by

$$\mathrm{JSR}(\mathcal{A}) := \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|P\|^{1/k}. \tag{1.2}$$

Here $\|\cdot\|$ denotes an arbitrary matrix norm.

This definition quantifies the maximal asymptotic growth rate of matrix products from $\mathcal{A}$. Theorem 1.3.2 below shows that the limit exists and does not depend on the chosen norm.

Illustrative Example

Consider the matrix set:

$$\mathcal{A} = \left\{ A_1 = \begin{bmatrix} 0.5 & 1 \\ 0 & 0.5 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0.3 & 0 \\ 1 & 0.3 \end{bmatrix} \right\}.$$

Each matrix individually has spectral radius less than 1. However, their product

$$A_1 A_2 = \begin{bmatrix} 0.5 & 1 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.3 & 0 \\ 1 & 0.3 \end{bmatrix} = \begin{bmatrix} 1.15 & 0.3 \\ 0.5 & 0.15 \end{bmatrix}$$

has spectral radius of approximately

$$\rho(A_1 A_2) \approx 1.2825 > 1.$$

This example shows that switching can amplify the effects of individual matrices, leading to divergent behavior even if all single matrices are contractive. Thus, the JSR can exceed the spectral radius of any individual matrix in the set.

The following results motivate the importance of bounding or approximating $\mathrm{JSR}(\mathcal{A})$ (Jungers, 2009):

- If $\mathcal{A}$ is irreducible, then the system (1.1) is *uniformly bounded*, meaning $\sup_t \|x_t\| < \infty$ for every $x_0 \in \mathbb{R}^d$, if and only if $\mathrm{JSR}(\mathcal{A}) \leq 1$.
- It is *uniformly asymptotically stable* if and only if $\mathrm{JSR}(\mathcal{A}) < 1$.

Understanding the joint spectral radius is therefore critical for analyzing the asymptotic behavior of switched linear systems and motivates reliable computational methods (Jungers, 2009). Beyond control theory, the JSR also appears in approximation theory and subdivision theory, with foundational references including stationary subdivision and two-scale refinement equations (Cavaretta et al., 1991; Daubechies and Lagarias, 1992; Maesumi, 2000). Related spectral-growth ideas are used in coding-theoretic capacity problems, including codes that avoid specified differences (Moision et al., 2001; Blondel et al., 2006), and in discrete mathematics and combinatorics (Protasov, 2000;

Jungers et al., 2008, 2009). These links motivate efficient computational methods, including the invariant-tree algorithm studied in this thesis (Mejstrik and Reif, 2025).

1.3. Fundamental Theoretical Results

Lemma 1.3.1 (Fekete's Lemma)**.** *Let $(a_n)_{n\geq 1}$ be a sequence of real, nonnegative numbers such that*

$$a_{m+n} \leq a_m + a_n \qquad \text{for all } m, n \geq 1. \tag{1.3}$$

Then the limit

$$\lim_{n\to\infty} \frac{a_n}{n}$$

exists and is equal to $\inf_{n\geq 1}\left(\frac{a_n}{n}\right)$.

Proof. Set $b_n := a_n/n$ and let $\alpha := \inf_{n\geq 1} b_n$. Fix $n \geq 1$ and write any $k \geq 1$ as $k = qn+r$ with integers $q \geq 0$ and $0 \leq r < n$. By subadditivity (1.3),

$$a_k = a_{qn+r} \leq qa_n + a_r.$$

Dividing by k gives

$$\frac{a_k}{k} \leq \frac{qa_n}{qn+r} + \frac{a_r}{qn+r} \leq \frac{a_n}{n} + \frac{a_r}{qn}.$$

Taking $\limsup_{k\to\infty}$ (equivalently $q \to \infty$) yields

$$\limsup_{k\to\infty} \frac{a_k}{k} \leq \frac{a_n}{n}.$$

Since n was arbitrary, we obtain

$$\limsup_{k\to\infty} \frac{a_k}{k} \leq \inf_{n\geq 1} \frac{a_n}{n} = \alpha.$$

On the other hand, by definition of infimum we always have $\alpha \leq a_k/k$ for all k, hence $\alpha \leq \liminf_{k\to\infty} a_k/k$. Therefore $\liminf = \limsup = \alpha$, so the limit exists and equals α. □

Theorem 1.3.2. *(Rota and Strang, 1960; Jungers, 2009) The JSR is well-defined and independent of the choice of the matrix norm.*

Proof. For a fixed matrix norm $\|\cdot\|$, define

$$M_k := \max_{P\in\mathcal{A}^k} \|P\| \quad \text{for } k \in \mathbb{N}.$$

By submultiplicativity, every product in $\mathcal{A}^{m+n}$ can be written as a product of one element of $\mathcal{A}^m$ and one element of $\mathcal{A}^n$, so

$$M_{m+n} \leq M_m M_n \quad \text{for all } m, n \in \mathbb{N}.$$

Hence the sequence $a_k := \log M_k$ is subadditive. Lemma 1.3.1 therefore implies that

$$\lim_{k\to\infty} \frac{a_k}{k}$$

exists, and thus so does

$$\lim_{k\to\infty} M_k^{1/k} = \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|P\|^{1/k}.$$

To prove independence of the norm, let $\|\cdot\|_1$ and $\|\cdot\|_2$ be two matrix norms on $\mathbb{R}^{d\times d}$. By norm equivalence in finite-dimensional vector spaces, there exist constants $c, C > 0$ such that

$$c\|P\|_1 \leq \|P\|_2 \leq C\|P\|_1 \quad \text{for all } P \in \mathbb{R}^{d\times d}.$$

Therefore

$$\max_{P\in\mathcal{A}^k} \|P\|_2^{1/k} \leq C^{1/k} \max_{P\in\mathcal{A}^k} \|P\|_1^{1/k}$$

and

$$\max_{P\in\mathcal{A}^k} \|P\|_1^{1/k} \leq c^{-1/k} \max_{P\in\mathcal{A}^k} \|P\|_2^{1/k}.$$

Passing to the limit as $k \to \infty$ shows that both norms yield the same value. □

To facilitate the analysis of the algorithms and presentation of the main results, we first establish fundamental properties of the JSR. The statements collected in this section are standard background results. When a proof is short and used later, we include it. Otherwise, we cite the literature and use the result as a reference point for subsequent chapters.

Homogeneity of the Joint Spectral Radius

Proposition 1.3.3. *(Jungers, 2009) The JSR is homogeneous, meaning for any set of matrices $\mathcal{A}$ and scalar $\alpha \in \mathbb{R}$ we have*

$$\mathrm{JSR}(\alpha\mathcal{A}) = |\alpha| \,\mathrm{JSR}(\mathcal{A}). \tag{1.4}$$

Proof. Let $\mathcal{A} = \{A_1, \ldots, A_n\}$ and let $\alpha \in \mathbb{R}$. For every product $P = A_{i_k} \cdots A_{i_1} \in \mathcal{A}^k$,

the corresponding product in $(\alpha\mathcal{A})^k$ is $\alpha^k P$. Hence

$$\begin{aligned}\mathrm{JSR}(\alpha\mathcal{A}) &= \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|\alpha^k P\|^{1/k} \\ &= |\alpha| \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|P\|^{1/k} \\ &= |\alpha|\,\mathrm{JSR}(\mathcal{A})\end{aligned}$$

□

Berger–Wang Characterization and Three-Member Inequality

Proposition 1.3.4. *(Berger and Wang, 1992) The* JSR *can equivalently be defined as*

$$\mathrm{JSR}(\mathcal{A}) = \limsup_{k\to\infty} \max_{P\in\mathcal{A}^k} \rho(P)^{1/k},$$

if $\mathcal{A}$ is a finite (or compact) set.

Proposition 1.3.5. *(Rota and Strang, 1960; Jungers, 2009) Let $\|\cdot\|$ be a matrix norm. Then for every $k \in \mathbb{N}$,*

$$\max_{P\in\mathcal{A}^k} \rho(P)^{1/k} \;\le\; \mathrm{JSR}(\mathcal{A}) \;\le\; \max_{P\in\mathcal{A}^k} \|P\|^{1/k}. \tag{1.5}$$

Proof. Fix $k \in \mathbb{N}$ and choose

$$P_{\max} \in \arg\max_{P\in\mathcal{A}^k} \rho(P)^{1/k}.$$

For the left-hand side we obtain, for any $r \in \mathbb{N}$,

$$\begin{aligned}\max_{P\in\mathcal{A}^k} \rho(P)^{1/k} &= \rho(P_{\max})^{1/k} = \rho(P_{\max})^{r/(kr)} \\ &= \rho(P_{\max}^r)^{1/(kr)} \le \max_{P\in\mathcal{A}^{kr}} \rho(P)^{1/(kr)} \\ &\le \limsup_{r\to\infty} \max_{P\in\mathcal{A}^{kr}} \rho(P)^{1/(kr)} \le \mathrm{JSR}(\mathcal{A}).\end{aligned} \tag{1.6}$$

For the right-hand side, set

$$M_n := \max_{P\in\mathcal{A}^n} \|P\| \qquad (n \in \mathbb{N}).$$

By submultiplicativity of the norm, every $Q \in \mathcal{A}^{m+n}$ can be written as $Q = RS$ with $R \in \mathcal{A}^m$ and $S \in \mathcal{A}^n$, hence

$$\|Q\| \le \|R\|\,\|S\| \le M_m M_n, \tag{1.7}$$

and taking maxima over $Q \in \mathcal{A}^{m+n}$ yields

$$M_{m+n} \leq M_m M_n \qquad \text{for all } m, n \in \mathbb{N}. \tag{1.8}$$

Define the real sequence $a_n := \log M_n$. Then from (1.8),

$$a_{m+n} = \log M_{m+n} \leq \log(M_m M_n) = \log M_m + \log M_n = a_m + a_n, \tag{1.9}$$

so $(a_n)_{n\geq 1}$ is subadditive. By Fekete's lemma, the limit $\lim_{n\to\infty} a_n/n$ exists and equals $\inf_{n\geq 1} a_n/n$. In particular, for the fixed k we have

$$\lim_{n\to\infty} \frac{a_n}{n} = \inf_{n\geq 1} \frac{a_n}{n} \leq \frac{a_k}{k}. \tag{1.10}$$

Exponentiating (1.10) gives

$$\begin{aligned} \lim_{n\to\infty} M_n^{1/n} &= \exp\left(\lim_{n\to\infty} \frac{a_n}{n}\right) \leq \exp\left(\frac{a_k}{k}\right) \\ &= \exp\left(\frac{\log M_k}{k}\right) = M_k^{1/k} \\ &= M_k^{1/k} = \max_{P\in\mathcal{A}^k} \|P\|^{1/k}. \end{aligned} \tag{1.11}$$

Since $\mathrm{JSR}(\mathcal{A}) = \lim_{n\to\infty} \max_{P\in\mathcal{A}^n} \|P\|^{1/n} = \lim_{n\to\infty} M_n^{1/n}$ by definition, the right-hand inequality in (1.5) follows. □

This inequality underlies many computational methods, since it yields computable lower and upper bounds on $\mathrm{JSR}(\mathcal{A})$. As $k \to \infty$, the upper bound $\max_{P\in\mathcal{A}^k} \|P\|^{1/k}$ converges to $\mathrm{JSR}(\mathcal{A})$. The lower bounds $\max_{P\in\mathcal{A}^k} \rho(P)^{1/k}$ need not converge, but their limit superior equals $\mathrm{JSR}(\mathcal{A})$ by Proposition 1.3.4.

Similarity Invariance and Reducibility

Proposition 1.3.6. *(Jungers, 2009) The* JSR *is invariant under similarity transformations, i.e., for any invertible matrix T we have*

$$\mathrm{JSR}(\mathcal{A}) = \mathrm{JSR}(T^{-1}\mathcal{A}T). \tag{1.12}$$

Proof. By Proposition 1.3.4,

$$\begin{aligned} \mathrm{JSR}(T^{-1}\mathcal{A}T) &= \limsup_{k\to\infty} \max_{I\in\mathcal{I}^k} \rho(T^{-1}A_I T)^{1/k} \\ &= \limsup_{k\to\infty} \max_{I\in\mathcal{I}^k} \rho(A_I)^{1/k} \\ &= \mathrm{JSR}(\mathcal{A}), \end{aligned}$$

because the spectral radius is invariant under similarity transformations. □

Definition 1.3.7. (Jungers, 2009) A set of matrices is called *(commonly) reducible* if there exists a nontrivial subspace of $\mathbb{R}^d$ that is invariant under all matrices in the set. Equivalently, there exists a change of basis that block-triangularizes all matrices in $\mathcal{A}$ simultaneously. If $\mathcal{A}$ is not reducible, it is called *irreducible*.

Proposition 1.3.8. *(Protasov, 1997) If $\mathcal{A}$ is reducible, then*

$$\mathrm{JSR}(\mathcal{A}) = \max\{\mathrm{JSR}(\mathcal{B}), \mathrm{JSR}(\mathcal{D})\}, \tag{1.13}$$

where $\mathcal{B} = \{B_1, \ldots, B_n\}$ and $\mathcal{D} = \{D_1, \ldots, D_n\}$ are the blocks of the block-triangularized matrices

$$T^{-1}A_jT = \begin{bmatrix} B_j & C_j \\ 0 & D_j \end{bmatrix}$$

under some invertible matrix T.

Example 1.3.9.

$$A_1 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

These matrices are reducible because there exists a nontrivial change of basis that transforms them into block-triangular form. The eigenvectors common to both matrices are

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Using the change of basis matrix

$$T = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix},$$

we compute the similarity transformations

$$T^{-1}A_1T = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}, \quad T^{-1}A_2T = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}.$$

Since both matrices are upper triangular in this basis, they share a common invariant subspace, proving that they are reducible. Interpreting the scalar blocks as one-dimensional matrix families, Proposition 1.3.8 yields

$$\mathrm{JSR}(\mathcal{A}) = \max\{\mathrm{JSR}(\{3, 5\}), \mathrm{JSR}(\{1, 1\})\} = 5$$

This decomposition can be applied iteratively until all resulting block families are irreducible. For the algorithmic chapters of the thesis, we may therefore restrict attention to irreducible families.

Finiteness Property and Spectrum-Maximizing Products

Definition 1.3.10. (Lagarias and Wang, 1995) A matrix set $\mathcal{A} = \{A_1, \ldots, A_n\}$ is said to possess the *finiteness property* if there exists a finite sequence of matrices $A_{i_1}, \ldots, A_{i_k}$ such that

$$\mathrm{JSR}(\mathcal{A}) = \rho(A_{i_k} \cdots A_{i_1})^{1/k}. \tag{1.14}$$

Any such product is referred to as a *spectrum-maximizing product* (s.m.p.).

The finiteness property does not hold for all matrix families; counterexamples were constructed in (Blondel et al., 2003). Nevertheless, spectrum-maximizing products remain central for practical algorithms, because many exact methods are built around promising candidate products.

Computational Complexity of JSR Computation

The computation of the joint spectral radius (JSR) is theoretically challenging due to several complexity results.

Approximating the JSR is NP-hard (Tsitsiklis and Blondel, 1997). Hence no polynomial-time algorithm is expected unless P = NP. Moreover, for general matrix families, even deciding whether the JSR is strictly less than one is undecidable (Blondel and Tsitsiklis, 2000). Despite these limitations, many structured instances still allow efficient numerical approximation.

S.m.p.-Candidates and Generators

The algorithms discussed later start from products that appear to maximize the averaged spectral radius. In the invariant-polytope setting these serve as s.m.p. candidates, while in the finite-tree and invariant-tree settings they also give rise to generators. Identifying such products is therefore a key preprocessing step.

Spectral Gap

Definition 1.3.11. (Mejstrik and Reif, 2025, Definition 1.9) A finite matrix family $\mathcal{A}$ is said to possess a *spectral gap* if there exists a constant $\gamma \in [0, 1)$ such that for every $k \in \mathbb{N}$ and every product $A_I \in \mathcal{A}^k$,

$$\rho(A_I) = \mathrm{JSR}(\mathcal{A})^k \quad \text{or} \quad \rho(A_I) \leq \gamma \ \mathrm{JSR}(\mathcal{A})^k.$$

Equivalently, every non-s.m.p. product is uniformly separated from the maximal growth rate by the factor $\gamma < 1$.

Remark 1.3.12. (Mejstrik and Reif, 2025; Guglielmi and Protasov, 2024) In the invariant-polytope setting, a spectral gap is a key finite-termination assumption. It is used together with uniqueness, up to cyclic permutations and powers, and simplicity of the leading eigenvalue of the s.m.p. candidate. Later chapters show where this gap condition enters the algorithmic termination analysis. We also note that (Guglielmi and Protasov, 2024) studies a related notion of gap as separation between the first and second Lyapunov exponents for matrix families.

Example 1.3.13. Consider

$$A_1 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \qquad A_2 = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \qquad \mathcal{A} = \{A_1, A_2\}.$$

Since $\max\{\rho(A_1), \rho(A_2)\} = 2$ and $\|A_1\|_\infty = \|A_2\|_\infty = 2$, we get $\mathrm{JSR}(\mathcal{A}) = 2$. Any product $A_I \in \mathcal{A}^k$ has the form

$$A_I = A_1^m A_2^{k-m} = \begin{bmatrix} 2^m & 0 \\ 0 & 2^{k-m} \end{bmatrix},$$

so

$$\rho(A_I) = 2^{\max\{m, k-m\}}.$$

If $m = 0$ or $m = k$, then $\rho(A_I) = 2^k = \mathrm{JSR}(\mathcal{A})^k$ and A_I is an s.m.p. product. If $1 \leq m \leq k-1$, then

$$\rho(A_I) \leq 2^{k-1} = \tfrac{1}{2} \ \mathrm{JSR}(\mathcal{A})^k.$$

Hence Definition 1.3.11 holds with $\gamma = \frac{1}{2}$.

1.4. Preprocessing and Normalization

All three algorithmic chapters use the same preprocessing pipeline:

We want to find the JSR of the finite set of matrices $\mathcal{A} = \{A_1, \cdots, A_n\}$.

1. **Assumptions**: $\mathcal{A}$ is irreducible.
2. **Candidates**: Find products $P = A_{i_k} \cdots A_{i_1}$ that maximize the averaged spectral radius
$$\lambda := \rho(P)^{1/k}.$$
3. **Rescaling**: Transform $\mathcal{A} \to \tilde{\mathcal{A}}$ with $\tilde{A}_i := \frac{1}{\lambda} A_i$.
4. **Verification**: Prove $\mathrm{JSR}(\tilde{\mathcal{A}}) \leq 1$. Together with the lower bound from the three-member inequality, this yields $\mathrm{JSR}(\tilde{\mathcal{A}}) = 1$. By homogeneity, this is equivalent to $\mathrm{JSR}(\mathcal{A}) = \lambda$.

The algorithms considered in the thesis differ only in the verification step. The invariant-polytope algorithm seeks a vector norm whose induced matrix norm already bounds the matrices from $\mathcal{A}$. The finite-tree algorithm instead decomposes products into factors bounded by 1 and one residual factor with only polynomial growth. The invariant-tree algorithm combines these two ideas.

2. Invariant-Polytope Algorithm

This chapter summarizes the invariant-polytope approach from (Guglielmi and Protasov, 2013) and related follow-up work. Its role is to provide the simplest termination mechanism used later in the thesis: one-step invariance of a balanced polytope under a rescaled matrix family.

2.1. Overview

2.1.1. Motivating Ideas

The invariant-polytope algorithm certifies an upper bound for the joint spectral radius by constructing a polytope norm that is adapted to the matrix family.

2.1.2. Problem Formulation

After preprocessing from Section 1.4, we obtain a rescaled family $\tilde{\mathcal{A}}$. From the chosen candidate product and the three-member inequality (1.5), we already know $\mathrm{JSR}(\tilde{\mathcal{A}}) \geq 1$. Hence it remains to prove $\mathrm{JSR}(\tilde{\mathcal{A}}) \leq 1$. To simplify notation, we write $\mathcal{A}$ for this rescaled family throughout the remainder of the chapter.

2.1.3. Core Idea

The algorithm tries to construct a balanced polytope $\mathcal{P} \subset \mathbb{R}^d$ that is invariant under $\mathcal{A}$. The associated Minkowski functional then yields a norm with respect to which all matrices in $\mathcal{A}$ have norm at most 1. This is the basic termination pattern of the thesis. Chapters 3 and 4 replace this direct invariance condition by more flexible tree-based bounds.

2.2. Algorithmic Formulation

2.2.1. Balanced Polytopes and Polytope Norms

Definition 2.2.1. A polytope $\mathcal{P} \subset \mathbb{R}^d$ is called *balanced* if it spans $\mathbb{R}^d$ and is equal to the symmetrized convex hull of a finite set of vertices. Thus there exists a finite set $V = \{v_1, \ldots, v_M\}$ such that

$$\mathcal{P} = \mathrm{co_s}(V) := \mathrm{co}(V, -V) = \left\{ x \in \mathbb{R}^d : x = \sum_{i=1}^{M} \alpha_i v_i \text{ and } \sum_{i=1}^{M} |\alpha_i| \leq 1 \right\}.$$

If $\mathcal{A}\mathcal{P} \subseteq \mathcal{P}$ for a family $\mathcal{A}$, then $\mathcal{P}$ is called *invariant under* $\mathcal{A}$.

Definition 2.2.2. (Magaril-Il'yaev and Tikhomirov, 2001) Let $C \subseteq \mathbb{R}^d$ be a nonempty convex set with $0 \in \mathrm{int}(C)$. The *Minkowski functional* associated to C is the mapping $p_C : \mathbb{R}^d \to [0, \infty)$ defined by

$$p_C(x) := \inf \{\lambda > 0 \,|\, x \in \lambda C\} \quad \text{for all } x \in \mathbb{R}^d.$$

Remark 2.2.3. If $\mathcal{P} = \mathrm{co_s}(V)$ is a balanced polytope, then the Minkowski functional defines a norm on $\mathbb{R}^d$. We denote this norm by

$$\|x\|_{\mathcal{P}} = \|x\|_{\mathrm{co_s}(V)} = \|x\|_V.$$

Moreover,

$$x \in \mathcal{P} \quad \Longleftrightarrow \quad \|x\|_{\mathcal{P}} \leq 1.$$

The induced matrix norm on $\mathbb{R}^{d \times d}$ is submultiplicative as well.

2.2.2. Algorithm Description

Algorithm 1 follows the real invariant-polytope construction from (Guglielmi and Protasov, 2013, Section 2). Starting from an initial finite set of vertices

$$V_0 = \{v_1, \ldots, v_M\},$$

each iteration forms the current balanced polytope

$$\mathcal{P}_i := \mathrm{co_s}(V_i).$$

We denote by $R_i \subseteq V_i$ the set of vertices that still have to be tested in iteration i. For every $v \in R_i$ and every matrix $A \in \mathcal{A}$, the algorithm tests whether Av already belongs to $\mathcal{P}_i$. By Remark 2.2.3, this is equivalent to testing whether $\|Av\|_{V_i} \leq 1$. If $Av \notin \mathcal{P}_i$,

then Av is added to the next vertex set. The next test set R_{i+1} consists exactly of the vertices added in iteration i. If $R_i = \emptyset$, then the algorithm terminates and returns $\mathcal{P}_i$.

2.2.3. Pseudocode

Algorithm 1 Invariant-polytope algorithm

Input: $\mathcal{A}$ (rescaled family of matrices), V_0 (set of starting vertices)
$R_0 \leftarrow V_0$
$i \leftarrow 0$
while $R_i \neq \emptyset$ **do**
  $V_{i+1} \leftarrow V_i$
  for $v \in R_i$ **do**
    $V_{i+1} \overset{\cup}{\leftarrow} \{Av : A \in \mathcal{A} \text{ and } \|Av\|_{\mathrm{co_s}(V_i)} > 1\}$
  $R_{i+1} \leftarrow V_{i+1} \setminus V_i$
  $i \leftarrow i + 1$
return $\mathrm{co_s}(V_i)$

2.2.4. Visualization

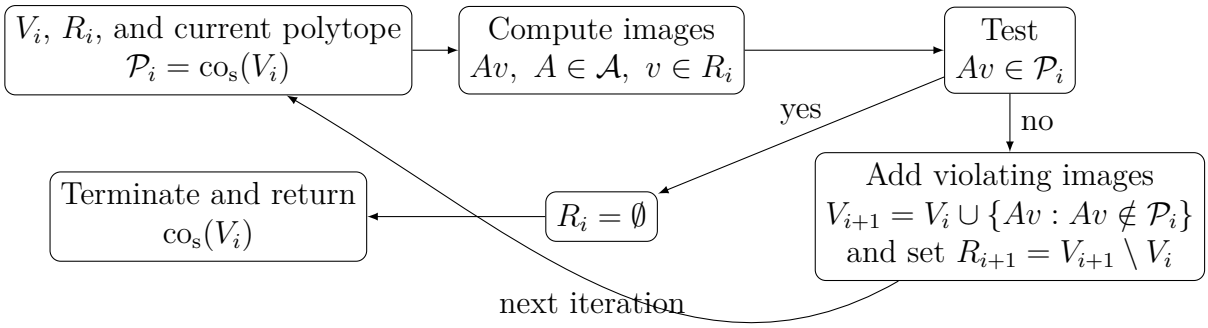

Figure 2.1.: Schematic iteration of the invariant-polytope algorithm. Termination occurs once all tested vertex images lie in the current polytope.

2.3. Termination Theory

2.3.1. Proof Strategy

The invariant-polytope argument is the simplest termination argument in the thesis. First, one shows that an invariant balanced polytope yields an extremal norm. Sec-

ond, one proves that algorithm termination implies exactly this invariance, because the algorithm checks all one-step images of the current vertices. Later chapters keep the same overall logic but replace one-step invariance by tree-based decompositions and polynomial bounds.

2.3.2. Auxiliary Results

Definition 2.3.1. (Protasov, 1996) A norm $\|\cdot\|$ on $\mathbb{R}^d$ is called *extremal* for a family of matrices $\mathcal{A}$ if

$$\max_{A\in\mathcal{A}}\|Ax\| \leq \mathrm{JSR}(\mathcal{A})\|x\| \quad \text{for all } x \in \mathbb{R}^d.$$

Theorem 2.3.2. *For every extremal norm $\|\cdot\|$ of a family of matrices $\mathcal{A}$,*

$$\max_{P\in\mathcal{A}^k}\|P\| = \mathrm{JSR}(\mathcal{A})^k$$

holds for every $k \in \mathbb{N}$.

Proof. We use the same symbol $\|\cdot\|$ for the vector norm and its induced matrix norm. Let $P = A_{i_k} \cdots A_{i_1} \in \mathcal{A}^k$. By submultiplicativity and Definition 2.3.1,

$$\|P\| \leq \prod_{j=1}^{k}\|A_{i_j}\| \leq \mathrm{JSR}(\mathcal{A})^k.$$

Taking the maximum over all products of length k gives

$$\max_{P\in\mathcal{A}^k}\|P\| \leq \mathrm{JSR}(\mathcal{A})^k.$$

The reverse inequality is the right-hand side of the three-member inequality (1.5), applied to the extremal norm:

$$\mathrm{JSR}(\mathcal{A}) \leq \max_{P\in\mathcal{A}^k}\|P\|^{1/k}.$$

Raising this estimate to the k-th power yields the opposite inequality. □

Remark 2.3.3. In particular, Theorem 2.3.2 with $k = 1$ yields

$$\max_{A\in\mathcal{A}}\|A\| = \mathrm{JSR}(\mathcal{A}).$$

Thus an extremal norm makes the joint spectral radius directly visible at the level of single matrices.

2.3.3. Main Result

Theorem 2.3.4. *(Guglielmi and Protasov, 2013, cf. Section 2) If a balanced polytope $\mathcal{P}$ is invariant under the rescaled family $\mathcal{A}$ from Section 1.4, then the Minkowski functional $\|\cdot\|_{\mathcal{P}}$ is an extremal norm for $\mathcal{A}$ and*

$$\mathrm{JSR}(\mathcal{A}) \leq 1.$$

Proof. Let $A \in \mathcal{A}$ and let $x \in \mathbb{R}^d$ satisfy $\|x\|_{\mathcal{P}} \leq 1$. By Remark 2.2.3, this means $x \in \mathcal{P}$. Since $\mathcal{P}$ is invariant under $\mathcal{A}$, we obtain $Ax \in \mathcal{P}$ and therefore

$$\|Ax\|_{\mathcal{P}} \leq 1.$$

By homogeneity, this implies

$$\|A\|_{\mathcal{P}} \leq 1 \quad \text{for all } A \in \mathcal{A}.$$

Because the family has been rescaled so that $\mathrm{JSR}(\mathcal{A}) \geq 1$, Definition 2.3.1 shows that $\|\cdot\|_{\mathcal{P}}$ is an extremal norm for $\mathcal{A}$. Applying Theorem 2.3.2 with $k = 1$, we obtain

$$\mathrm{JSR}(\mathcal{A}) = \max_{A \in \mathcal{A}} \|A\|_{\mathcal{P}} \leq 1.$$

□

2.3.4. Algorithmic Termination

Theorem 2.3.5. *If Algorithm 1 terminates and returns $\mathcal{P} = \mathrm{co_s}(V)$, then $\mathcal{P}$ is invariant under $\mathcal{A}$. Consequently,*

$$\mathrm{JSR}(\mathcal{A}) \leq 1.$$

Proof. At termination, every tested image already belongs to the returned polytope. Hence

$$Av \in \mathcal{P} \quad \text{for all } A \in \mathcal{A} \text{ and } v \in V.$$

Now take an arbitrary $x \in \mathcal{P}$. By Definition 2.2.1, there exist coefficients $\alpha_1, \ldots, \alpha_M$ such that

$$x = \sum_{i=1}^{M} \alpha_i v_i \quad \text{and} \quad \sum_{i=1}^{M} |\alpha_i| \leq 1.$$

Therefore, for every $A \in \mathcal{A}$,

$$Ax = \sum_{i=1}^{M} \alpha_i A v_i.$$

Since $\mathcal{P}$ is symmetric and each vector Av_i belongs to $\mathcal{P}$, the vector $\operatorname{sign}(\alpha_i)Av_i$ belongs to $\mathcal{P}$ as well. Because $0 \in \mathcal{P}$ and $\sum_{i=1}^{M} |\alpha_i| \leq 1$, convexity yields

$$Ax = \sum_{i=1}^{M} |\alpha_i| \operatorname{sign}(\alpha_i) Av_i + \left(1 - \sum_{i=1}^{M} |\alpha_i|\right) 0 \in \mathcal{P}.$$

Thus $Ax \in \mathcal{P}$ for every $x \in \mathcal{P}$ and every $A \in \mathcal{A}$, which proves $\mathcal{A}\mathcal{P} \subseteq \mathcal{P}$. The conclusion follows from Theorem 2.3.4. □

Remark 2.3.6. Theorem 2.3.5 explains why termination certifies the chosen candidate scaling. A separate question is when finite termination actually occurs. For the real invariant-polytope algorithm, sharp finite-termination criteria are tied to dominant products with unique simple leading eigenvalues; see (Guglielmi and Protasov, 2013, Theorem 4). Later refinements extend the method to several competing candidates and better conditioned initial data (Guglielmi and Protasov, 2016; Mejstrik, 2020).

2.4. Implementation Details

2.4.1. Stopping Criterion and Candidate Rejection

For a candidate product

$$\Pi = A_{d_k} \cdots A_{d_1},$$

the real invariant-polytope algorithm is formulated under the assumption that Π has a unique simple leading eigenvalue. Let v_1 be a corresponding leading eigenvector and let v_1^* be the leading eigenvector of Π^*, normalized by

$$(v_1, v_1^*) = 1.$$

For the cyclic permutations of Π, one obtains corresponding dual leading eigenvectors v_j^*. The stopping criterion from (Guglielmi and Protasov, 2013, Section 2.4) then says: if during the iteration a tested image Av satisfies

$$\|Av\|_{\mathrm{co_s}(V)} > 1$$

and there exists an index j such that

$$|(v_j^*, Av)| > 1,$$

then the chosen candidate cannot be an s.m.p. and must be rejected.

2.4.2. Rebalancing and Additional Initial Vertices

The basic algorithm is most transparent when there is a single dominant candidate. In practical problems, however, several nearly maximizing products may compete, and the initial polytope may be badly conditioned. Later work therefore introduces balancing of several candidate roots and additional initial vertices in order to improve conditioning and finite-termination behavior (Guglielmi and Protasov, 2016; Mejstrik, 2020). These refinements are not needed for the basic correctness proof above, but they are important in implementations and form part of the background for the hybrid chapter.

2.5. Remarks

2.5.1. Complex Variants

If the leading eigenvector associated with a candidate product is complex, then a real balanced-polytope norm is no longer sufficient and must be replaced by a complex-balanced polytope norm. This changes the convex-hull membership test, while the high-level structure of the algorithm remains the same.

In this thesis, we restrict attention to the real case. For details on complex variants, we refer to (Guglielmi et al., 2005; Guglielmi and Zennaro, 2007; Guglielmi and Protasov, 2013).

Remark 2.5.1. Finite termination of the invariant-polytope algorithm is not guaranteed for every irreducible family. Even when extremal norms exist, they need not be polyhedral, and in that case the algorithm need not terminate with an exact certificate. It may still provide useful bounds, but it need not construct a finite invariant polytope (Guglielmi and Protasov, 2013).

3. Finite-Tree Algorithm

This chapter summarizes the finite-tree approach from (Möller and Reif, 2014). Its role is to provide the first tree-based termination argument in the thesis and to bridge the invariant-polytope method from Chapter 2 to the hybrid method in Chapter 4.

3.1. Overview

3.1.1. Motivating Ideas

The finite-tree algorithm bounds arbitrary products $P \in \mathcal{A}^k$ by decomposing them into factors that are 1-bounded with respect to a chosen norm and one residual factor that grows at most polynomially.

3.1.2. Problem Formulation

After preprocessing from Section 1.4, we obtain a rescaled family $\tilde{\mathcal{A}}$. From the chosen candidate product and the three-member inequality (1.5), we already know $\mathrm{JSR}(\tilde{\mathcal{A}}) \geq 1$. Hence it remains to prove $\mathrm{JSR}(\tilde{\mathcal{A}}) \leq 1$. To simplify notation, we write $\mathcal{A}$ for this rescaled family throughout the remainder of the chapter.

3.1.3. Core Idea and Notation

The finite-tree argument is the first generalization of the invariant-polytope strategy from Chapter 2. Instead of requiring one-step invariance of a balanced polytope, it organizes products in a finite tree and proves that every long product can be decomposed into leaf factors bounded by 1 and one remaining factor with only polynomial growth.

We begin with the rooted product tree $T_{\mathcal{A}}$ whose nodes are all positive multi-indices. For the empty index, we set

$$A_{[]} = I_d,$$

and this node is the root.

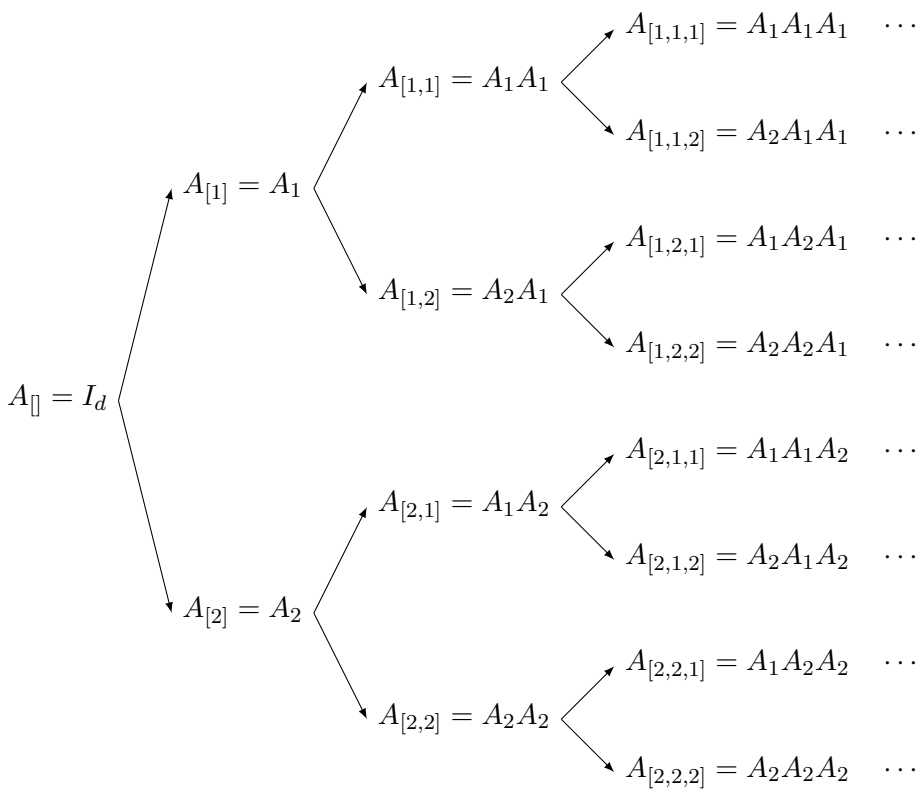

Figure 3.1.: Example of the product tree $T_{\mathcal{A}}$ for $\mathcal{A} = \{A_1, A_2\}$.

In general, a direct search in $T_{\mathcal{A}}$ is not feasible. Following (Möller and Reif, 2014), one therefore introduces generators and set-valued nodes. We fix a generator set

$$\mathcal{G} = \{G_1, \dots, G_m\},$$

where each generator satisfies

$$G_r \in \mathcal{A}^{k_r} \quad \text{and} \quad \rho(G_r)^{1/k_r} \leq 1.$$

Definition 3.1.1. (Möller and Reif, 2014) Set

$$\mathcal{J} := \{-m, \dots, -1, 1, \dots, n\}$$

and define

$$A_{-r} := G_r \quad \text{for } r = 1, \dots, m.$$

For a multi-index $J = [j_1, \dots, j_\ell] \in \mathcal{J}^\ell$, the associated set of matrix products is

$$A_J := \left\{ A_{j_\ell}^{e_\ell} \cdots A_{j_1}^{e_1} : \begin{array}{l} e_s = 1 \text{ if } j_s > 0, \\ e_s \in \mathbb{N}_0 \text{ if } j_s < 0 \end{array} \right\}.$$

Thus positive entries represent single matrices from $\mathcal{A}$, while negative entries represent

arbitrary powers of generators.

Definition 3.1.2. (Möller and Reif, 2014) A finite rooted tree T is called an $(\mathcal{A}, \mathcal{G})$*-tree* if its nodes are multi-indices from $\bigcup_{\ell \in \mathbb{N}_0} \mathcal{J}^\ell$, the root is the empty index $[]$, and every node is one of the following:

- a leaf,
- a parent of exactly n positive children,
- or a parent with at least one generator child.

Definition 3.1.3. (Möller and Reif, 2014) A node of an $(\mathcal{A}, \mathcal{G})$-tree is called *covered* if its associated product set is contained in the product set of one of its ancestors. Otherwise, it is called *uncovered*. The set of uncovered leaves is denoted by

$$\mathcal{L}(T) := \{K \in T : K \text{ is an uncovered leaf}\}$$

and is called the *leafage* of T.

Definition 3.1.4. (Möller and Reif, 2014) An $(\mathcal{A}, \mathcal{G})$-tree T is called 1-*bounded* with respect to a matrix norm $\|\cdot\|$ if

$$\sup\{\|P\| : L \in \mathcal{L}(T),\ P \in A_L\} \leq 1.$$

If

$$\sup\{\|P\| : L \in \mathcal{L}(T),\ P \in A_L\} < 1,$$

then T is called *strictly* 1-*bounded*.

3.2. Algorithmic Formulation

3.2.1. Algorithm Description

The finite-tree algorithm performs a non-exhaustive search in the space of $(\mathcal{A}, \mathcal{G})$-trees. It starts from the positive children of the root. We denote by Q_i the set of active nodes that still have to be tested in iteration i. For each active node $J \in Q_i$, the algorithm checks whether the associated product set A_J is already 1-bounded. If not, the node is expanded according to a child-selection rule, and the new children are placed into the next active set Q_{i+1}. If $Q_i = \emptyset$, then the current tree is a finite certificate candidate.

Algorithm 2 Finite-tree algorithm

Input: $\mathcal{A}$ (rescaled family of matrices), $\mathcal{G}$ (set of generators)
$T_0 \leftarrow \{[], [1], \ldots, [n]\}$
$Q_0 \leftarrow \{[1], \ldots, [n]\}$
$i \leftarrow 0$
while $Q_i \neq \emptyset$ **do**
  $T_{i+1} \leftarrow T_i$
  $Q_{i+1} \leftarrow \emptyset$
  for $J \in Q_i$ **do**
    if $\sup\{\|P\| : P \in A_J\} > 1$ **then**
      $\mathcal{C} \leftarrow \textsc{selectChildren}(J, \mathcal{A}, \mathcal{G})$
      $T_{i+1} \overset{\cup}{\leftarrow} \{[J, j] : j \in \mathcal{C}\}$
      $Q_{i+1} \overset{\cup}{\leftarrow} \{[J, j] : j \in \mathcal{C}\}$
  $i \leftarrow i + 1$
return T_i

3.2.2. Pseudocode

3.2.3. Visualization

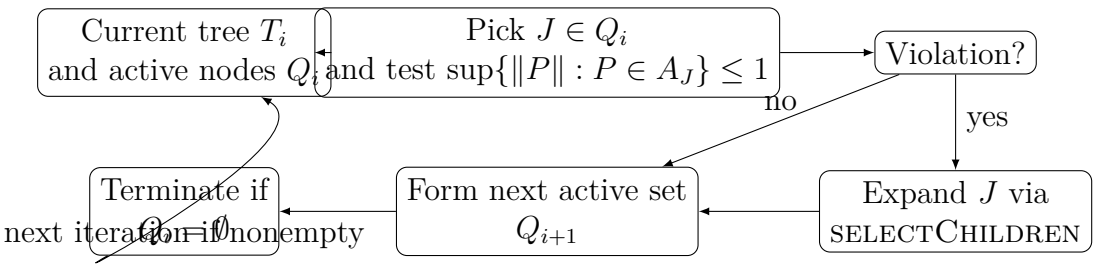

Figure 3.2.: Workflow of the finite-tree algorithm. Termination occurs when no active node violates the 1-bound.

3.3. Termination Theory

3.3.1. Proof Strategy

The finite-tree argument is the tree-based analogue of the invariant-polytope argument from Chapter 2. Instead of proving one-step invariance of a set, it bounds arbitrary products by repeated prefix-leaf decomposition. The proof has two steps. First, one proves a polynomial bound for products encoded by nodes of a fixed finite $(\mathcal{A}, \mathcal{G})$-tree. Second, one decomposes an arbitrary product into leaf factors bounded by 1 and one residual node factor, then passes to the k-th root limit in the definition of JSR.

3.3.2. Auxiliary Results

Lemma 3.3.1. *For every finite $(\mathcal{A}, \mathcal{G})$-tree T there exists a monotone polynomial p such that for every node $K \in T$ and every product $P \in \mathcal{A}^k$ encoded by K, one has*

$$\|P\| \leq p(k).$$

Proof. Let $K = [j_1, \ldots, j_\ell]$ be a fixed node of T. For each position $r = 1, \ldots, \ell$, define a monotone polynomial q_r as follows. If $j_r > 0$, then the corresponding factor is always the single matrix A_{j_r}, so we set

$$q_r(t) := \|A_{j_r}\| \quad \text{for all } t \in \mathbb{N}_0.$$

If $j_r < 0$, then $A_{j_r} = G_{-j_r}$ is a generator with spectral radius at most 1. Hence the powers of A_{j_r} grow at most polynomially by the Jordan-normal-form argument, so there exists a monotone polynomial q_r such that

$$\|A_{j_r}^e\| \leq q_r(e) \quad \text{for all } e \in \mathbb{N}_0.$$

Set

$$p_K := q_\ell \cdots q_1$$

and

$$p := \sum_{K \in T} p_K.$$

Since T is finite, the function p is a monotone polynomial.

Now let

$$P = A_{j_\ell}^{e_\ell} \cdots A_{j_1}^{e_1} \in \mathcal{A}^k$$

be encoded by the node K. By submultiplicativity,

$$\|P\| \leq \|A_{j_\ell}^{e_\ell}\| \cdots \|A_{j_1}^{e_1}\| \leq q_\ell(e_\ell) \cdots q_1(e_1).$$

Every exponent satisfies $e_r \leq k$, because the encoded product has total length k. Using the monotonicity of the polynomials q_r, we obtain

$$\|P\| \leq q_\ell(k) \cdots q_1(k) = p_K(k) \leq p(k).$$

Since the node K and the product P were arbitrary, the lemma follows. □

Lemma 3.3.2. *(Möller and Reif, 2014, cf. Lemma 4.1 and the proof of Theorem 3.3) Let T be a finite $(\mathcal{A}, \mathcal{G})$-tree and let $I \in \mathcal{I}^k$. Then there exist unique multi-indices L and P, each either positive or equal to $[]$, such that*

$$I = [L, P] \quad \textit{and hence} \quad A_I = A_P A_L.$$

Moreover, either $P = []$ and A_I is encoded by a node of T, or A_L is encoded by a leaf in $\mathcal{L}(T)$ and $|P| < k$.

Proof. Let

$$\mathcal{I}_*(T) := \left\{ J \in \bigcup_{r \in \mathbb{N}_0} \mathcal{I}^r : A_J \in \bigcup_{K \in T} A_K \right\}$$

denote the set of positive multi-indices encoded by nodes of T. Choose the longest prefix L of I that belongs to $\mathcal{I}_*(T)$, and let P be the complementary suffix, so that $I = [L, P]$. The maximality of the prefix makes this choice unique.

If $P = []$, then the whole product A_I is encoded by a node of T, and the first case holds. Assume therefore that $P \neq []$. Let K be maximal among the nodes of T whose encoded product set contains A_L. By maximality of the prefix L, the node K cannot have a positive child continuing the word I. It also cannot have a generator child, because every generator child contains the power 0 and would therefore still encode A_L at a deeper node. Finally, the argument in (Möller and Reif, 2014, proof of Lemma 4.1) excludes the possibility that K is covered. Hence K is an uncovered leaf, that is, $K \in \mathcal{L}(T)$. Therefore A_L is encoded by a leaf of T. Since $P \neq []$, one also has $|P| < k$. □

3.3.3. Main Result

Theorem 3.3.3. *(Möller and Reif, 2014, Theorem 3.3) If the rescaled family $\mathcal{A}$ admits a finite 1-bounded $(\mathcal{A}, \mathcal{G})$-tree, then*

$$\mathrm{JSR}(\mathcal{A}) \leq 1.$$

Proof. Take an arbitrary positive multi-index $I \in \mathcal{I}^k$. Repeatedly apply Lemma 3.3.2 to the current remainder until the remaining factor is encoded by a node of T. This yields multi-indices $I_1, \ldots, I_r$ such that

$$A_I = A_{I_r} \cdots A_{I_1},$$

where each factor A_{I_j} with $j = 1, \ldots, r-1$ is encoded by a leaf in $\mathcal{L}(T)$ and the last factor A_{I_r} is encoded by a node of T.

By Definition 3.1.4, the leaf factors satisfy

$$\|A_{I_j}\| \leq 1 \quad \text{for } j = 1, \ldots, r-1.$$

By Lemma 3.3.1, there exists a monotone polynomial p such that

$$\|A_{I_r}\| \leq p(k).$$

Therefore,

$$\|A_I\| = \|A_{I_r} \cdots A_{I_1}\| \leq \|A_{I_r}\| \cdots \|A_{I_1}\| \leq p(k).$$

Since the multi-index I was arbitrary,

$$\mathrm{JSR}(\mathcal{A}) = \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|P\|^{1/k} \leq \lim_{k\to\infty} p(k)^{1/k} = 1.$$

□

Remark 3.3.4. Combined with the preprocessing step from Section 1.4, Theorem 3.3.3 certifies that the chosen candidate scaling is correct. In this sense, the finite-tree method plays the same role for tree-based certificates that Theorem 2.3.5 plays for invariant polytopes.

3.3.4. Termination Conditions

Theorem 3.3.3 gives the central sufficient condition: a finite 1-bounded $(\mathcal{A}, \mathcal{G})$-tree implies the desired upper bound for the rescaled family. For actual finite termination of the search procedure, the child-selection rule must lead to such a tree after finitely many expansions.

3.4. Implementation Details

3.4.1. Child-Selection Policy

The function selectChildren decides whether a violating node is expanded by all positive children or by a suitable collection of generator children. This choice is part of the practical search strategy and follows the rules discussed in (Möller and Reif, 2014).

3.5. Remarks

Remark 3.5.1. (Möller and Reif, 2014, Theorem 3.5) If the rescaled family $\mathcal{A}$ is irreducible and has a spectral gap at 1 in the sense of Definition 1.3.11, then, with a suitable choice of norm, a finite tree of the type required in Theorem 3.3.3 exists. Thus the finite-tree method is guaranteed to succeed in that setting.

Remark 3.5.2. (Möller and Reif, 2014) Generators are essential for the strength of the method. They make it possible to treat infinite families of products through finite tree nodes and can substantially reduce the depth of the searched tree. This is one of the key differences between the finite-tree and invariant-polytope approaches.

4. Invariant-Tree Algorithm

This chapter contains the thesis-specific contribution: a detailed formulation of the invariant-tree algorithm and a corresponding termination analysis, based on the hybrid framework of (Mejstrik and Reif, 2025). In the recent literature, the same construction is also referred to as the *tree-flavoured-invariant-polytope-algorithm*. Implementation details for finite 1-bounding of generated product sets are taken from the cited work and are not claimed as original contributions here.

4.1. Overview

4.1.1. Combination Strategies

At its core, the invariant-polytope algorithm constructs a polytope norm adapted to the matrix family, while the finite-tree algorithm controls growth through product decompositions. This naturally suggests combining both algorithms.

One option is modular: first run the invariant-polytope algorithm for several iterations, then use the resulting norm inside the finite-tree phase. A second option is concurrent: update norm information and tree-decomposition information in each iteration. Following (Mejstrik and Reif, 2025), we use the concurrent variant.

4.1.2. Problem Formulation

After preprocessing from Section 1.4, we obtain a scaled family $\tilde{\mathcal{A}}$. From the chosen candidate and the three-member inequality (1.5), we already know $\mathrm{JSR}(\tilde{\mathcal{A}}) \geq 1$. Hence proving $\mathrm{JSR}(\tilde{\mathcal{A}}) \leq 1$ is sufficient. To simplify notation, we write $\mathcal{A}$ for this rescaled family throughout the remainder of the chapter.

As in Chapter 3, it is therefore enough to prove a polynomial product bound. More precisely, we show that there exist a polynomial p and a norm $\|\cdot\|$ such that

$$\|P\| \leq p(k) \quad \text{for all } P \in \mathcal{A}^k,\ k \in \mathbb{N}.$$

The polynomial bound is obtained by decomposing each multi-index into finitely many parts arising from 1-v-bounded test trees, plus one residual part controlled by finite-tree arguments.

4.2. Algorithmic Formulation

The hybrid algorithm uses a vertex-wise version of the 1-boundedness concept from Chapter 3. This is the local analogue of the V-closedness criterion from (Mejstrik and Reif, 2025, Definition 2.4 and Theorem 2.7).

Definition 4.2.1. Let $v \in \mathbb{R}^d$ and let $\|\cdot\|$ be a vector norm on $\mathbb{R}^d$. An $(\mathcal{A}, \mathcal{G})$-tree T is called 1-v-*bounded* with respect to $\|\cdot\|$ if

$$\sup_{L \in \mathcal{L}(T)} \|Lv\| \leq 1.$$

If

$$\sup_{L \in \mathcal{L}(T)} \|Lv\| < 1,$$

then T is called *strictly* 1-v-*bounded*.

If $\mathcal{P} = \mathrm{co_s}(V)$ is the polytope generated by the final vertex set, then, by Remark 2.2.3,

$$\|Lv\|_{\mathcal{P}} \leq 1 \quad \Longleftrightarrow \quad Lv \in \mathcal{P}.$$

Hence a family of trees $(T_v)_{v \in V}$ that is 1-v-bounded for every $v \in V$ is the vertex-wise counterpart of a V-closed tree in the sense of (Mejstrik and Reif, 2025).

The algorithm starts, as in the invariant-polytope method, from a set V_0 of starting vertices. Each vertex set V_i defines a vector polytope norm $\|\cdot\|_{V_i}$. We denote by $R_i \subseteq V_i$ the set of vertices that still have to be tested in iteration i. For every vertex $v \in R_i$, we construct a test tree T_v. If some L in the leafage of T_v satisfies $\|Lv\|_{V_i} > 1$, equivalently $Lv \notin \mathrm{co_s}(V_i)$, then Lv is added to the next vertex set. The next test set R_{i+1} consists exactly of the vertices added in iteration i. If $R_i = \emptyset$, then the algorithm terminates. This is the exact closedness test from (Mejstrik and Reif, 2025, Algorithm 3.1). The proof of termination only needs the weaker conclusion $\|Lv\|_V \leq 1$ for the final polytope.

4.2.1. Pseudocode

Algorithm 3 is adapted from (Mejstrik and Reif, 2025, Algorithm 3.1).

Algorithm 3 Invariant-tree algorithm

Input: $\mathcal{A}$ (rescaled family of matrices), $\mathcal{G}$ (set of generators), V_0 (set of starting vertices)
$R_0 \leftarrow V_0$
$i \leftarrow 0$
while $R_i \neq \emptyset$ **do**
  $V_{i+1} \leftarrow V_i$
  for $v \in R_i$ **do**
    Construct a test tree T_v according to the current tree-selection rule
    $V_{i+1} \overset{\cup}{\leftarrow} \{Lv : L \in \mathcal{L}(T_v) \text{ and } \|Lv\|_{V_i} > 1\}$
  $R_{i+1} \leftarrow V_{i+1} \setminus V_i$
  $i \leftarrow i + 1$
return $\mathrm{co_s}(V_i)$

4.3. Termination Theory

Theorem 4.3.1. *If Algorithm 3 terminates, then* $\mathrm{JSR}(\mathcal{A}) \leq 1$.

4.3.1. Proof Strategy

The following argument is the vertex-wise analogue of (Mejstrik and Reif, 2025, Theorem 2.7). Suppose the algorithm terminates after K iterations, and write

$$V = \bigcup_{i=0}^{K} V_i.$$

We equip this final vertex set with the associated polytope norm $\|\cdot\|_V$. The proof has two steps: first, we obtain a polynomial bound for $A_I v$ with $v \in V$ by combining 1-v-boundedness with prefix-leaf decomposition. Second, we pass from the vertices to arbitrary unit vectors in $\mathrm{co_s}(V)$ by using the balanced-polytope representation from Definition 2.2.1.

4.3.2. Auxiliary Termination Results

Lemma 4.3.2. *If Algorithm 3 terminates, there exists a monotone polynomial p such that for every product $P \in \mathcal{A}^k$ and $v \in V$, one has*

$$\|Pv\|_{\mathrm{co_s}(V)} \leq p(k).$$

Proof. If the algorithm terminates, then the set of vertices $V = \{v_1, \ldots, v_M\}$ is finite and for each $v \in V$ there exists a finite $(\mathcal{A}, \mathcal{G})$-tree T_v that is 1-v-bounded with respect

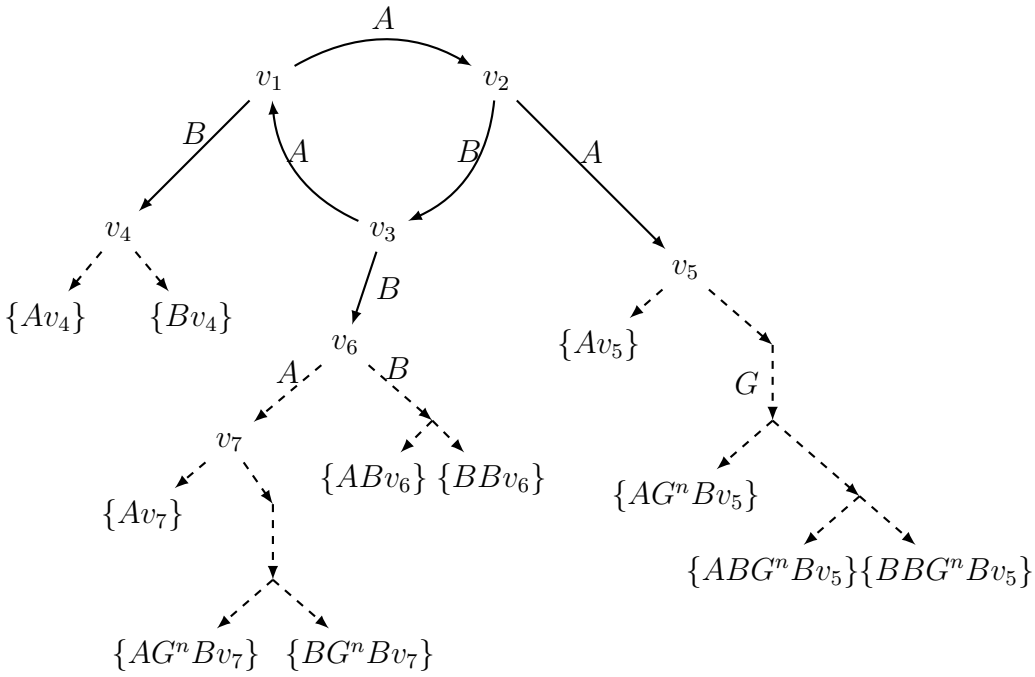

Figure 4.1.: Cyclic tree structure generated by the algorithm. Here $\mathcal{A}$ consists of two matrices A and B and the chosen candidate is $\Pi = ABA$. Dashed arrows indicate $(\mathcal{A}, \mathcal{G})$-trees used for bounds. Solid arrows indicate vertices added to V in one iteration. Leaf labels with powers G^n represent matrix-vector products that remain 1-v-bounded for all $n \in \mathbb{N}_0$.

to $\|\cdot\|_{\mathrm{co_s}(V)}$. For every leaf $K \in \mathcal{L}(T_v)$ and every positive multi-index L encoded by K, one has

$$\|A_L v\|_{\mathrm{co_s}(V)} \leq 1.$$

Hence, by Definition 2.2.1, there exist coefficients $\lambda_1, \ldots, \lambda_M$ such that

$$A_L v = \sum_{j=1}^{M} \lambda_j v_j \quad \text{and} \quad \sum_{j=1}^{M} |\lambda_j| \leq 1.$$

Moreover, Lemma 3.3.1 provides a monotone polynomial p_v such that

$$\|Pv\|_{\mathrm{co_s}(V)} \leq p_v(k)$$

for every product $P \in \mathcal{A}^k$ encoded by a node of T_v. Set

$$p := 1 + \sum_{v \in V} p_v.$$

We prove by induction on the length of I that

$$\|A_I v\|_{\mathrm{co_s}(V)} \leq p(k)$$

holds for every positive multi-index $I \in \mathcal{I}^k$ and every $v \in V$, and also for the empty index $I = []$. For $I = []$, we have $A_{[]} = I_d$ and therefore $\|A_{[]} v\|_{\mathrm{co_s}(V)} = \|v\|_{\mathrm{co_s}(V)} \leq 1 \leq p(0)$. Now let $k \geq 1$ and assume the claim has already been proved for all shorter products. If A_I is encoded by a node of T_v, then

$$\|A_I v\|_{\mathrm{co_s}(V)} \leq p_v(k) \leq p(k).$$

Assume therefore that A_I is not encoded by a node of T_v. By Lemma 3.3.2, applied to the tree T_v, there exist positive multi-indices P and L such that

$$A_I = A_P A_L,$$

where A_L is encoded by a leaf of T_v and $|P| < k$. Writing

$$A_L v = \sum_{j=1}^{M} \lambda_j v_j \quad \text{with} \quad \sum_{j=1}^{M} |\lambda_j| \leq 1,$$

we obtain

$$\begin{aligned}\|A_I v\|_{\mathrm{co_s}(V)} &= \left\| A_P \sum_{j=1}^{M} \lambda_j v_j \right\|_{\mathrm{co_s}(V)} \\ &\leq \sum_{j=1}^{M} |\lambda_j| \|A_P v_j\|_{\mathrm{co_s}(V)} \\ &\leq p(|P|) \sum_{j=1}^{M} |\lambda_j| \\ &\leq p(k).\end{aligned} \tag{4.1}$$

where the third line uses the induction hypothesis and the last line uses the monotonicity of p. This proves the claim and hence the lemma. □

4.3.3. Main Termination Result

Proof. Let $V = \bigcup_{i=0}^{K} V_i$ be the final vertex set produced by the algorithm, and set $\mathcal{P} := \mathrm{co_s}(V)$. By Lemma 4.3.2, there exists a monotone polynomial p such that

$$\|A_I v_j\|_{\mathcal{P}} \leq p(k) \quad \text{for all } I \in \mathcal{I}^k \text{ and } v_j \in V.$$

Now take an arbitrary $x \in \mathbb{R}^d$ with $\|x\|_{\mathcal{P}} = 1$. By Definition 2.2.1, there exist coefficients $\mu_1, \ldots, \mu_M$ such that

$$x = \mu_1 v_1 + \ldots + \mu_M v_M \quad \text{and} \quad \sum_{j=1}^{M} |\mu_j| \leq 1.$$

Therefore,

$$\|A_I x\|_{\mathcal{P}} \leq \sum_{j=1}^{M} |\mu_j| \|A_I v_j\|_{\mathcal{P}} \leq p(k) \sum_{j=1}^{M} |\mu_j| \leq p(k).$$

Hence

$$\|A_I\|_{\mathcal{P}} = \sup_{\|x\|_{\mathcal{P}}=1} \|A_I x\|_{\mathcal{P}} \leq p(k).$$

By Remark 2.2.3, the induced matrix norm $\|\cdot\|_{\mathcal{P}}$ is submultiplicative. Consequently,

$$\mathrm{JSR}(\mathcal{A}) = \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|P\|_{\mathcal{P}}^{1/k} \leq \lim_{k\to\infty} p(k)^{1/k} = 1.$$

□

Remark 4.3.3. If each test tree T_v consists only of the root and its positive children, then Algorithm 3 reduces to Algorithm 1. This is the simplest V-closed-tree configuration from (Mejstrik and Reif, 2025, Theorem 2.8), so the invariant-tree algorithm genuinely extends the invariant-polytope viewpoint.

Remark 4.3.4. Assume Algorithm 3 terminates and there exists a single test tree T such that $T_v = T$ for all $v \in V$. Then the hybrid argument collapses to the fixed-tree argument of Chapter 3. Repeated application of Lemma 3.3.2 yields a factorization

$$A_I = A_{I_r} \cdots A_{I_1},$$

where each factor A_{I_j} with $j = 1, \ldots, r-1$ is encoded by a leaf of T, and the last factor A_{I_r} is encoded by a node of T. Thus the leaf factors are bounded by 1, while the last factor is bounded polynomially by Lemma 3.3.1. In this special case, no vertex-wise recursion is needed.

4.4. Implementation Considerations

Following the smallest-tree heuristic proposed in (Mejstrik and Reif, 2025, Section 3.2 and Figure 3), we use the following terminology.

Definition 4.4.1. Fix a matrix family $\mathcal{A}$ and a generator G. Among all $(\mathcal{A}, \mathcal{G})$-trees that have a generator branch labeled by G at the root, contain a first covered node, and satisfy the structural conditions from Definition 3.1.2, we call a tree *minimal* if its leafage is minimal among trees of this form. A fixed choice of such a tree is denoted by $T_{(G,\mathcal{A})}$.

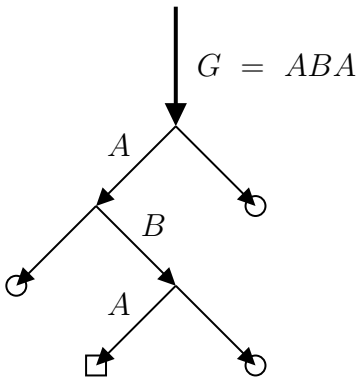

Figure 4.2.: Minimal tree structure $T_{(G,\mathcal{A})}$ for the candidate $G = ABA$ and $\mathcal{A} = \{A, B\}$. The arrows marked with circles are uncovered leaves, and the arrow marked with a square is a covered leaf.

In principle, the invariant-tree algorithm allows a different test tree for each vertex $v \in V$, as long as the chosen tree satisfies the $(\mathcal{A}, \mathcal{G})$-tree conditions from Definition 3.1.2.

In this thesis, we restrict the implementation to fixed minimal choices. For each vertex, this keeps the generator branch and the first covered-node effect while minimizing the leafage that has to be tested. It therefore reduces the number of checks of the form $\|Lv\|_{\mathrm{co_s}(V)}$ and serves as the default tree-selection policy.

Because generator branches encode families such as $\{G^r : r \in \mathbb{N}_0\}$, a naive implementation would require infinitely many tests. Following (Mejstrik and Reif, 2025, Section 3.3), we split the generator dynamics into a leading-eigenspace part and a stable remainder and then estimate the stable remainder by means of an ε-pseudospectral radius. Concretely, for a matrix Δ and $\varepsilon > 0$, the ε-pseudospectral radius is

$$\rho_\varepsilon(\Delta) := \sup\Big\{|z| : \|(zI - \Delta)^{-1}\| > \varepsilon^{-1}\Big\}.$$

It yields bounds of the form

$$\|\Delta^n\| \le \varepsilon^{-1}\rho_\varepsilon(\Delta)^{n+1},$$

so one chooses ε in such a way that $\rho_\varepsilon(\Delta) < 1$. This turns the infinite generator branch into finitely many verifiable conditions for the current vertex.

The practical implementation also uses a maximal iteration counter $r_{\max}$, meaning that at most $r_{\max}$ outer iterations are performed. If no certificate is found before reaching this threshold, the algorithm stops and reports the current bounds for the original, unscaled family. We denote the current lower bound by $\underline{\rho}$ and the current upper bound by $\overline{\rho}$. Here $\underline{\rho}$ is obtained from the best candidate product found in preprocessing, while $\overline{\rho}$ denotes the best upper bound currently available from the right-hand side of the three-member inequality together with the current norm or tree data. Thus the output is an interval

$$\underline{\rho} \;\le\; \mathrm{JSR}(\mathcal{A}) \;\le\; \overline{\rho}.$$

In practice, these bounds can already be accurate even when full termination is not achieved.

From a computational perspective, the dominant costs are tree construction and the leafage checks $\|Lv\|_{\mathrm{co_s}(V)}$. Restricting to minimal trees keeps the tested leafage small. Generator branches add the finite verification procedure from (Mejstrik and Reif, 2025, Section 3.3). No global polynomial-time bound in (n, d) is expected in full generality, which is consistent with the known hardness results for JSR computation (Tsitsiklis and Blondel, 1997; Blondel and Tsitsiklis, 2000).

Finally, candidate updates from the preprocessing stage in Section 1.4 can still be combined with invariant-tree iterations to improve robustness in practice.

4.5. Remarks

4.5.1. Polytope Invariance

After Algorithm 3 terminates, the polytope $\mathrm{co_s}(V)$ is, in general, not invariant under the full family $\mathcal{A}$. This is expected, since the invariant-tree algorithm enforces polynomial product bounds rather than one-step invariance under each matrix in $\mathcal{A}$.

The termination certificate is first vertex-wise. This is the local version of the V-closedness concept from (Mejstrik and Reif, 2025, Definition 2.4). For every $v \in V$, the algorithm provides a test tree T_v such that each leaf matrix in $\mathcal{L}(T_v)$ maps v back into the polytope. Equivalently, with

$$\Gamma_V := \{(v, L) : v \in V,\ L \in \mathcal{L}(T_v)\},$$

one has

$$(v, L) \in \Gamma_V \quad \Longrightarrow \quad Lv \in \mathrm{co_s}(V).$$

A stronger, uniform statement is obtained if one can construct a single $(\mathcal{A}, \mathcal{G})$-tree $T_\star$ that bounds all vertices simultaneously, i.e.

$$\|Lv\|_{\mathrm{co_s}(V)} \leq 1 \quad \text{for all } v \in V \text{ and } L \in \mathcal{L}(T_\star).$$

Then $T_\star$ induces leafage invariance on the whole polytope:

$$x \in \mathrm{co_s}(V),\ L \in \mathcal{L}(T_\star) \quad \Longrightarrow \quad Lx \in \mathrm{co_s}(V).$$

Indeed, writing $x = \sum_{i=1}^{M} \alpha_i v_i$ with $\sum_{i=1}^{M} |\alpha_i| \leq 1$ and $v_i \in V$, one gets

$$Lx = \sum_{i=1}^{M} \alpha_i L v_i,$$

and each term Lv_i belongs to $\mathrm{co_s}(V)$ by assumption. By convexity and symmetry of $\mathrm{co_s}(V)$, this implies $Lx \in \mathrm{co_s}(V)$.

The existence and efficient construction of such a global tree $T_\star$ from the family $(T_v)_{v \in V}$ is a separate algorithmic problem and remains part of future work.

This invariance interpretation, together with the intended play on words relative to the invariant-polytope algorithm, motivates the name *invariant-tree algorithm*.

5. Numerical Testing

This chapter compares the invariant-polytope algorithm (Algorithm 1), the finite-tree algorithm (Algorithm 2), and the invariant-tree algorithm (Algorithm 3) with respect to iteration counts, runtime, and certificate quality.

5.1. Experimental Setup

This chapter is intended to document the computational evidence for the thesis. In its final form, it should make clear which matrix families were tested, how the algorithms were implemented, and which quantitative criteria were used for the comparison.

All experiments should be reported with the same preprocessing pipeline from Section 1.4. In particular, the candidate-selection strategy, the scaling convention, the stopping tolerances, and the maximal iteration counter should be kept fixed unless a section explicitly studies their effect.

The numerical comparison should at least record the following quantities:

- whether the algorithm terminated with an exact certificate,
- the number of outer iterations until termination or stopping,
- the wall-clock runtime,
- the size of the produced certificate object, such as the number of vertices or tested leaves,
- and, in the non-terminating case, the interval $[\underline{\rho}, \overline{\rho}]$ returned for the JSR.

5.2. Generated Matrix Families

This section should compare the three algorithms on generated matrix families across increasing dimensions. The main purpose is to document how the computational cost changes with dimension and to what extent the invariant-tree algorithm preserves solvability when the baseline methods begin to fail.

Table 5.1.: Experimental setup. Replace the placeholder entries by the actual implementation details used for the reported experiments.

Item	Value to insert
Programming language / environment	–
Software version	–
Hardware / CPU	–
Stopping tolerance	–
Maximal iteration counter $r_{\max}$	–
Candidate-selection strategy	–

Table 5.2.: Benchmark families used in the numerical tests. Replace the placeholders by the actual families and sample sizes.

Family class	Dimension(s)	Number of instances	Section
Generated random families	–	–	Section 5.2
Families with unique s.m.p.	–	–	Section 5.3
Families with multiple s.m.p.s	–	–	Section 5.4

The discussion in the final version should answer at least the following questions. How does the termination rate depend on the matrix dimension? How quickly do runtime and certificate size grow? At what dimensions do the three algorithms begin to separate clearly in performance?

Table 5.3.: Generated matrix families. Replace the placeholder entries by aggregated numerical results such as median runtime, median iteration count, and termination rate.

Dimension	Instances	Algorithm	Termination rate	Median iterations	Median runtime
–	–	Invariant polytope	–	–	–
–	–	Finite tree	–	–	–
–	–	Invariant tree	–	–	–

After inserting the data, this section should end with a short interpretation of the dominant trend. In particular, note whether the invariant-tree algorithm mainly improves termination, runtime, or both, and explain where the computational bottleneck appears to move.

5.3. Instances with a Unique s.m.p.

This section should report comparisons on benchmark families with a unique s.m.p. In this regime the invariant-polytope algorithm is already expected to perform well, so the

Insert a plot for the generated-family experiment here.
Suggested options: runtime versus dimension, termination rate versus dimension, or certificate size versus dimension.

Figure 5.1.: Placeholder for the generated-family comparison plot.

main question is whether the invariant-tree algorithm remains competitive when the problem is favorable to the classical method.

The final discussion should compare the algorithms not only by runtime, but also by iteration count and certificate size. If the invariant-tree algorithm is slower on these easy instances, this section should say so explicitly and quantify the gap.

Table 5.4.: Benchmark instances with a unique s.m.p. Replace the placeholders by the measured quantities for each benchmark family.

Instance	Dimension	Algorithm	Terminated	Iterations	Runtime	Certificate size
–	–	Invariant polytope	–	–	–	–
–	–	Finite tree	–	–	–	–
–	–	Invariant tree	–	–	–	–

Insert a figure for the unique-s.m.p. benchmark comparison here.
Suggested options: grouped runtime bars, iteration counts, or a scatter plot comparing the invariant-tree and invariant-polytope methods.

Figure 5.2.: Placeholder for the unique-s.m.p. benchmark comparison.

When this section is completed, it should close with a concise statement explaining whether the hybrid method merely matches the classical algorithm on these instances or whether it also brings a practical advantage.

5.4. Instances with Multiple s.m.p.s

This section should compare the effective solve space for multiple-s.m.p. instances. This is the part of the numerical chapter where the invariant-tree algorithm is expected to show its main advantage, so the final write-up should make the difference in solvability especially clear.

Besides reporting termination and runtime, this section should state what kind of certificate was obtained when exact termination failed. In particular, it should report how narrow the resulting interval $[\underline{\rho}, \overline{\rho}]$ was and whether the invariant-tree method still produced a useful bound where the other methods stalled.

The final paragraph of this section should identify the main qualitative conclusion from the benchmark set. For example, it should state whether the invariant-tree algorithm

Table 5.5.: Benchmark instances with multiple s.m.p.s. Replace the placeholders by the measured output of the three algorithms.

Instance	Dimension	Algorithm	Terminated	Runtime	Lower bound	Upper bound
–	–	Invariant polytope	–	–	–	–
–	–	Finite tree	–	–	–	–
–	–	Invariant tree	–	–	–	–

Insert a figure for the multiple-s.m.p. comparison here.
Suggested options: solvability map, interval-width comparison, or runtime bars on the difficult benchmark set.

Figure 5.3.: Placeholder for the multiple-s.m.p. benchmark comparison.

enlarges the class of solvable examples, whether it mainly improves certificate quality, or whether its benefit is concentrated in a narrower subclass of problems.

At the end of the chapter, add a short transitional paragraph summarizing the numerical evidence in a way that can be reused in Chapter 6.

6. Conclusions and Outlook

This final chapter collects the main conclusions of the thesis. It separates numerical observations, resolved problem classes, remaining limitations, and directions for future research.

6.1. Summary of Invariant-Tree Numerics

This section should distill the numerical evidence from Chapter 5 into a small number of clear statements. In the final version, it should summarize how the invariant-tree algorithm compares with the invariant-polytope and finite-tree algorithms in terms of termination behavior, runtime, and certificate quality.

The most useful structure is to state the main observation first and then support it with one short paragraph on each of the following points: easy instances with a unique s.m.p., difficult instances with several competing products, and the overall runtime trade-off introduced by the hybrid method.

6.2. Resolved Problem Classes

This section should identify the classes of matrix families for which the thesis now provides a satisfactory algorithmic picture. In particular, it should state where the invariant-polytope algorithm already gives a complete answer, where the finite-tree argument is sufficient, and where the invariant-tree algorithm extends the range of exact certificates.

When you fill this section, it is helpful to phrase the result class by class. For example, one paragraph can cover families with a unique simple s.m.p., a second can cover classes with a useful spectral gap, and a third can discuss instances where vertex-wise trees succeed although a single invariant polytope is difficult to construct.

6.3. Remaining Open Problem Classes

This section should state the limits of the present approach just as clearly as its successes. In particular, it should mention which benchmark families remain difficult, whether the bottleneck lies in candidate selection, tree construction, generator-branch verification, or numerical conditioning, and where the present termination theory still falls short of a full characterization.

The final version should also separate theoretical limitations from implementation limitations. If a problem class is believed to be tractable in principle but is not yet covered by the current implementation, that distinction should be stated explicitly.

6.4. Directions for Future Research

This section should turn the open issues above into concrete research directions. The most convincing structure is to move from the most immediate implementation problems to the broader mathematical questions suggested by the thesis.

6.4.1. Optimization of Tree Choice and Vertex Selection

This subsection should discuss how the current heuristic tree selection could be improved. In particular, it should address whether one can construct better test trees than the minimal trees used in this thesis, how vertex ordering influences runtime, and whether local tree choices can be coordinated more globally.

It is also a natural place to mention the open problem raised in Chapter 4: how to build a single global tree from the family of vertex-wise trees, or how to decide efficiently when such a global tree exists.

6.4.2. Extension to the Complex Setting

This subsection should explain why the real-valued restriction is mathematically convenient but algorithmically limiting. The final version should indicate what would have to change in the invariant-polytope, finite-tree, and invariant-tree parts of the framework in order to treat complex leading eigenvectors and complex-balanced polytopes.

6.4.3. Optimization for Nonnegative Matrix Families

This subsection should discuss the special structure of nonnegative matrix families. If the numerical tests contain such examples, this is the place to connect them to possible simplifications of the tree search, polytope membership tests, and generator verification. It should also state whether the invariant-tree approach seems especially promising in that setting.

The chapter should end with a short final paragraph that returns to the thesis goal from Chapter 1: the invariant-tree algorithm is intended as a bridge between invariant-polytope certificates and tree-based product decompositions. The final sentence should state clearly what this thesis has achieved on that path and what remains to be done.

A. Notation Glossary

A.1. Constants and Globally Defined Variables

A_i Matrix in the family $\mathcal{A}$.

d Dimension of each matrix $A_i \in \mathbb{R}^{d \times d}$.

n Number of matrices in $\mathcal{A}$.

$\mathcal{A} = \{A_1, \ldots, A_n\}$ Base set of matrices.

$\tilde{\mathcal{A}} = \{\tilde{A}_1, \ldots, \tilde{A}_n\}$ Scaled matrix set after preprocessing.

m Number of generators.

$\mathcal{G} = \{G_1, \ldots, G_m\}$ Set of generators.

k Length of a product.

α Scalar.

λ Eigenvalue.

x Vector.

ρ Spectral radius.

σ Singular value.

JSR Joint spectral radius operator.

A.2. Products and Multi-Indices

$\mathcal{I} = \{1, \ldots, n\}$ Positive matrix-index set.

$\mathcal{I}^k$ Set of positive multi-indices of length k.

i Positive index.

I Positive multi-index of length k, i.e. $I = [i_1, \ldots, i_k] \in \mathcal{I}^k$.

$[]$ Empty multi-index of length 0.

$A_{[]} = I_d$ Product encoded by the empty multi-index.

$A_I = A_{[i_1,\dots,i_k]} = A_{i_k} \cdots A_{i_1}$ Product encoded by I.

$\mathcal{J} = \{-m, \dots, -1, 1, \dots, n\}$ Extended index set including generator indices.

$\mathcal{J}^k$ Set of extended multi-indices of length k.

j Extended index.

J Extended multi-index of length k, i.e. $J = [j_1, \dots, j_k] \in \mathcal{J}^k$.

$A_J = A_{[j_1,\dots,j_k]}$ Product set encoded by J.

A.3. Norms and JSR

$\|\cdot\|_{\mathrm{cos}(V)} = \|\cdot\|_V$ Polytope norm notation used for matrices and vectors.

$\mathrm{JSR}(\mathcal{A}) := \lim_{k\to\infty} \max_{P\in\mathcal{A}^k} \|P\|^{1/k}$ Norm-based definition of JSR.

$\mathrm{JSR}(\mathcal{A}) = \limsup_{k\to\infty} \max_{P\in\mathcal{A}^k} \rho(P)^{1/k}$ Berger–Wang characterization.

A.4. Algorithmic Notation

Invariant-polytope algorithm Baseline algorithm from Chapter 2.

Finite-tree algorithm Tree-based algorithm from Chapter 3.

Invariant-tree algorithm Combined algorithm from Chapter 4.

M Number of vertices in a finite vertex set representation $V = \{v_1, \dots, v_M\}$.

V_i Set of vertices available after iteration i.

R_i Set of vertices that remain to be tested in iteration i.

V Union of vertex sets over all steps.

T_i Current finite tree in iteration i of the finite-tree algorithm.

Q_i Active nodes that remain to be tested in iteration i of the finite-tree algorithm.

T_v Test tree attached to the vertex v in the invariant-tree algorithm.

$\overset{\cup}{\leftarrow}$ Appending by union in pseudocode.

A.5. Tree Terminology

T Finite rooted tree used to encode product families.

Root Node without a predecessor.

Child and parent If a node $t \in T$ has predecessor $p \in T$, then t is a child of p and p is the parent of t.

Leaf Node without children.

Internal node Node with at least one child.

$T_{\mathcal{A}}$ Infinite product tree generated by $\mathcal{A}$.

$(\mathcal{A}, \mathcal{G})$-**tree** Tree whose nodes are extended multi-indices and whose internal nodes have either n positive children or at least one generator child.

Covered or uncovered node A node is covered if its encoded product set is contained in that of an ancestor; otherwise it is uncovered.

$\mathcal{L}(T)$ Leafage of T, i.e. all uncovered leaves.

1-**bounded** Every product encoded by a leaf in $\mathcal{L}(T)$ has norm at most 1 in a chosen matrix norm.

1-v-**bounded** Every product encoded by a leaf in $\mathcal{L}(T)$ maps a fixed vector v to a vector of norm at most 1.

PL decomposition Prefix-leaf decomposition.

$T_{(G,\mathcal{A})}$ Minimal tree associated with the generator G and the family $\mathcal{A}$.

Bibliography

Alizadeh, F. and Goldfarb, D. (2003). Second-order cone programming. *Mathematical Programming*, 95(1):3–51.

Andersen, E. D., Roos, C., and Terlaky, T. (2003). On implementing a primal-dual interior-point method for conic quadratic optimization. *Mathematical Programming*, 95(2):249–277.

Ando, T. and Shih, M.-H. (1998). Simultaneous contractibility. *SIAM Journal on Matrix Analysis and Applications*, 19(2):487–498.

Barabanov, N. E. (1988). Lyapunov indicator for discrete inclusions, i–iii. *Automation and Remote Control*, 49(2):152–157.

Berger, M. A. and Wang, Y. (1992). Bounded semigroups of matrices. *Linear Algebra and its Applications*, 166:21–27.

Berstel, J. (2005). Growth of repetition-free words—a review. *Theoretical Computer Science*, 340(2):280–290.

Blondel, V. D., Jungers, R. M., and Protasov, V. Y. (2006). On the complexity of computing the capacity of codes that avoid forbidden difference patterns. *IEEE Transactions on Information Theory*, 52(11):5122–5127.

Blondel, V. D. and Nesterov, Y. (2005). Computationally efficient approximations of the joint spectral radius. *SIAM Journal on Matrix Analysis and Applications*, 27(1):256–272.

Blondel, V. D., Nesterov, Y., and Theys, J. (2005). On the accuracy of the ellipsoid norm approximation of the joint spectral radius. *Linear Algebra and its Applications*, 394:91–107.

Blondel, V. D., Theys, J., and Vladimirov, A. A. (2003). An elementary counterexample to the finiteness conjecture. *SIAM Journal on Matrix Analysis and Applications*, 24(4):963–970.

Blondel, V. D. and Tsitsiklis, J. N. (2000). Approximating the spectral radius of sets of matrices in the max-algebra is NP-hard. *IEEE Transactions on Automatic Control*, 45(9):1762–1765.

Cassaigne, J. (1993). Counting overlap-free binary words. In *STACS 93*, volume 665 of *Lecture Notes in Computer Science*, pages 216–225, Berlin. Springer.

Cavaretta, A. S., Dahmen, W., and Micchelli, C. A. (1991). *Stationary Subdivision*. Number 453 in Memoirs of the American Mathematical Society. American Mathematical Society.

Cicone, A., Guglielmi, N., Serra-Capizzano, S., and Zennaro, M. (2010). Finiteness property of pairs of 2
*times*2 sign-matrices via real extremal polytope norms. *Linear Algebra and its Applications*, 432(2–3):796–816.

Daubechies, I. and Lagarias, J. C. (1992). Two-scale difference equations. II. local regularity, infinite products of matrices and fractals. *SIAM Journal on Mathematical Analysis*, 23(4):1031–1079.

Elsner, L. (1995). The generalized spectral-radius theorem: an analytic-geometric proof. *Linear Algebra and its Applications*, 220:151–159.

Finch, S. (2008). Private communication. Private communication.

Finch, S., Sebah, P., and Bai, Z.-Q. (2008). Odd entries in pascal's trinomial triangle.

Fornasini, E. and Valcher, M. E. (2010). Stabilizability of discrete-time positive switched systems. In *Proceedings of the 49th IEEE Conference on Decision and Control (CDC 2010)*, pages 432–437.

Goldwasser, J., Klostermeyer, W., Mays, M., and Trapp, G. (1999). The density of ones in pascal's rhombus. *Discrete Mathematics*, 204:231–236.

Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD.

Gripenberg, G. (1996). Computing the joint spectral radius. *Linear Algebra and its Applications*, 234:43–60.

Guglielmi, N., Manni, C., and Vitale, D. (2011). Convergence analysis of c^2 hermite interpolatory subdivision schemes by explicit joint spectral radius formulas. *Linear Algebra and its Applications*, 434:784–902.

Guglielmi, N. and Protasov, V. Y. (2013). Exact computation of joint spectral characteristics of linear operators. *Foundations of Computational Mathematics*, 13(1):37–97.

Guglielmi, N. and Protasov, V. Y. (2016). Invariant polytopes of linear operators with applications to regularity of wavelets and of subdivisions. *SIAM Journal on Matrix Analysis and Applications*, 37(1):18–52.

Guglielmi, N. and Protasov, V. Y. (2024). Computing the spectral gap of a family of matrices. *Mathematics of Computation*, 93(345):259–291.

Guglielmi, N., Wirth, F., and Zennaro, M. (2005). Complex polytope extremality results for families of matrices. *SIAM Journal on Matrix Analysis and Applications*, 27(3):721–743.

Guglielmi, N. and Zennaro, M. (2007). Balanced complex polytopes and related vector and matrix norms. *Journal of Convex Analysis*, 14:729–766.

Guglielmi, N. and Zennaro, M. (2008). An algorithm for finding extremal polytope norms of matrix families. *Linear Algebra and its Applications*, 428(10):2265–2282.

Guglielmi, N. and Zennaro, M. (2009). Finding extremal complex polytope norms for families of real matrices. *SIAM Journal on Matrix Analysis and Applications*, 31(2):602–620.

Gurvits, L. (1995). Stability of discrete linear inclusions. *Linear Algebra and its Applications*, 231:47–85.

Hare, K. G., Morris, I. D., Sidorov, N., and Theys, J. (2011). An explicit counterexample to the lagarias–wang finiteness conjecture. *Advances in Mathematics*, 226(6):4667–4701.

Hassan, M. F., Ivrissimitzis, I. P., Dodgson, N. A., and Sabin, M. A. (2002). An interpolating 4-point c^2 ternary stationary subdivision scheme. *Computer Aided Geometric Design*, 19(1):1–18.

Hechler, J., Mößner, B., and Reif, U. (2009). c^1-continuity of the generalized four-point scheme. *Linear Algebra and its Applications*, 430(11–12):3019–3029.

Horn, R. A. and Johnson, C. R. (1985). *Matrix Analysis*. Cambridge University Press, Cambridge.

Jungers, R. M. (2009). *The Joint Spectral Radius: Theory and Applications*, volume 385 of *Lecture Notes in Control and Information Sciences*. Springer, Berlin.

Jungers, R. M. and Protasov, V. Y. (2009). Counterexamples to the complex polytope extremality conjecture. *SIAM Journal on Matrix Analysis and Applications*, 31(2):404–409. Preprint: arXiv:0802.2654.

Jungers, R. M., Protasov, V. Y., and Blondel, V. D. (2008). Efficient algorithms for deciding the type of growth of products of integer matrices. *Linear Algebra and its Applications*, 428(10):2296–2312.

Jungers, R. M., Protasov, V. Y., and Blondel, V. D. (2009). Overlap-free words and spectra of matrices. *Theoretical Computer Science*, 410(38–40):3670–3684.

Kozyakin, V. (2005). A dynamical systems construction of a counterexample to the finiteness conjecture. In *Proceedings of the 44th IEEE Conference on Decision and Control and ECC 2005*, pages 2338–2343.

Kozyakin, V. S. (1990). Algebraic unsolvability of problem of absolute stability of desynchronized systems. *Automation and Remote Control*, 51(6):754–759.

Kozyakin, V. S. (2009). On the computational aspects of the theory of joint spectral radius. *Doklady Mathematics*, 80(1):487–491.

Lagarias, J. C. and Wang, Y. (1995). The finiteness conjecture for the generalized spectral radius of a set of matrices. *Linear Algebra and its Applications*, 214.

Maesumi, M. (1996). An efficient lower bound for the generalized spectral radius. *Linear Algebra and its Applications*, 240:1–7.

Maesumi, M. (2000). Joint spectral radius and hölder regularity of wavelets. *Computers & Mathematics with Applications*, 40(1):145–155.

Maesumi, M. (2008). Optimal norms and the computation of joint spectral radius of matrices. *Linear Algebra and its Applications*, 428(10):2324–2338.

Magaril-Il'yaev, G. G. and Tikhomirov, V. M. (2001). *Convex Analysis: Theory and Applications*, volume 222 of *Translations of Mathematical Monographs*. American Mathematical Society. Translated from Russian by Dmitry Chibisov.

Mason, O. and Shorten, R. N. (2007). Quadratic and copositive lyapunov functions and the stability of positive switched linear systems. In *Proceedings of the 2007 American Control Conference (ACC 2007)*, pages 657–662, New York City, NY, USA. IEEE. Marriott Marquis Hotel at Times Square, July 11–13, 2007.

Mejstrik, T. (2020). Improved invariant polytope algorithm and applications. *ACM Transactions on Mathematical Software*, 46(3):29:1–29:26.

Mejstrik, T. and Reif, U. (2025). A hybrid approach to joint spectral radius computation. *Linear Algebra and its Applications*.

Mengi, E. (2006). *Measures for Robust Stability and Controllability*. PhD thesis, New York University.

Moision, B. E., Orlitsky, A., and Siegel, P. H. (2001). On codes that avoid specified differences. *IEEE Transactions on Information Theory*, 47(1):433–442.

Möller, C. (2015). *A New Strategy for Exact Determination of the Joint Spectral Radius*. PhD thesis, TU Darmstadt.

Möller, C. and Reif, U. (2014). A tree-based approach to joint spectral radius determination. *Linear Algebra and its Applications*, 463:154–170.

Parrilo, P. A. and Jadbabaie, A. (2008). Approximation of the joint spectral radius using sum of squares. In *Linear Algebra and its Applications*, volume 428, pages 2385–2402.

Protasov, V. Y. (1996). The joint spectral radius and invariant sets of linear operators. *Fundamentalnaya i Prikladnaya Matematika*, 2(1):205–231.

Protasov, V. Y. (1997). The generalized spectral radius. a geometric approach. *Izvestiya: Mathematics*, 61(5):995–1030.

Protasov, V. Y. (2000). Asymptotic behaviour of the partition function. *Sbornik: Mathematics*, 191(3–4):381–414.

Protasov, V. Y. (2004). On the regularity of de rham curves. *Izvestiya: Mathematics*, 68(3):27–68.

Protasov, V. Y. (2010). Invariant functionals for random matrices. *Functional Analysis and Its Applications*, 44(3):230–233.

Protasov, V. Y., Jungers, R. M., and Blondel, V. D. (2010). Joint spectral characteristics of matrices: a conic programming approach. *SIAM Journal on Matrix Analysis and Applications*, 31(4):2146–2162.

Reznick, B. (1990). Some binary partition functions. In *Analytic Number Theory: Proceedings of a Conference in Honor of P.T. Bateman*, pages 451–477. Birkhäuser, Boston.

Rota, G.-C. and Strang, G. (1960). A note on the joint spectral radius. *Indagationes Mathematicae*, 22(4):379–381.

Rudin, W. (1976). *Principles of Mathematical Analysis*. McGraw-Hill, New York, 3 edition.

Stewart, G. W. and Sun, J. G. (1990). *Matrix Perturbation Theory*. Academic Press, New York.

Strang, G. (2001). The joint spectral radius (commentary). In *Collected Works of Gian-Carlo Rota*. Birkhäuser, Basel. Commentary on Rota–Strang (1960).

Tsitsiklis, J. N. and Blondel, V. D. (1997). The lyapunov exponent and joint spectral radius of pairs of matrices are hard—when not impossible—to compute and to approximate. *Mathematics of Control, Signals and Systems*, 10(1):31–40.

Vagnoni, C. and Zennaro, M. (2009). The analysis and the representation of balanced complex polytopes in 2D. *Foundations of Computational Mathematics*, 9(3):259–294.

Vandergraft, J. S. (1968). Spectral properties of matrices which have invariant cones. *SIAM Journal on Applied Mathematics*, 16:1208–1222.

Villemoes, L. (1994). Wavelet analysis of refinement equations. *SIAM Journal on Mathematical Analysis*, 25(5):1433–1460.

Zheng, H., Zhao, H., Ye, Z., and Zhou, M. (2007). Differentiability of a 4-point ternary subdivision scheme and its application. *IAENG International Journal of Applied Mathematics*, 36(1):19–24.